

Data Analysis with Python

Lesson 4: Data Cleaning Techniques

Data Cleaning Techniques - Study Notes

Key Concepts

1. **Missing Value Handling** – Strategies to identify, quantify, and impute or remove gaps in data.
2. **Duplicate Detection & Removal** – Techniques to spot identical or near duplicate rows and decide when to drop or consolidate them.
3. **Type Conversion & Normalization** – Converting data into consistent, analysis ready types (e.g., dates, categorical codes, numeric scales).
4. **Data Validation & Integrity Checks** – Rules to ensure values fall within expected ranges or match reference tables.
5. **Automation & Reproducibility** – Using scripts or pipelines to apply cleaning steps consistently across datasets.

Summary

Data cleaning is the foundational step that guarantees the reliability of any subsequent analysis. Missing values can bias results, while duplicates inflate sample sizes and distort statistics. Converting data types—such as turning strings that represent dates into `datetime` objects—enables proper sorting, grouping, and mathematical operations. A systematic approach involves first exploring the dataset, then applying targeted techniques: imputation (mean, median, mode, or model based), deletion, or flagging; deduplication based on key columns; and type casting with validation rules. Automating these steps in a reproducible pipeline (e.g., using pandas, dplyr, or Spark) ensures consistency across projects and eases collaboration.

Important Points

- **Missing Data:**
 - `NaN` in pandas, `NULL` in SQL, or empty strings in CSVs.
 - Use `df.isna().sum()` to quantify missingness.
 - Impute with `df.fillna()` or advanced methods (`sklearn.impute.SimpleImputer`, `IterativeImputer`).
 - Consider dropping rows/columns when missingness exceeds a threshold (e.g., >70%).
- **Duplicate Rows:**
 - Detect with `df.duplicated()` or `df.duplicated(subset=[cols])`.
 - Decide to drop (`df.drop_duplicates()`) or consolidate (e.g., averaging numeric columns).
 - Use `keep='first'/'last'/'False'` to control which duplicate to retain.
- **Type Conversion:**
 - Convert strings to dates: `pd.to_datetime(df['date_col'])`.
 - Convert numeric strings: `pd.to_numeric(df['num_col'], errors='coerce')`.

- Convert categorical codes: `df['cat_col'] = df['cat_col'].astype('category')`.
- Always validate: `df['col'].unique()` after conversion.
 - **Validation Rules:**
- Range checks: `df[(df['age'] < 0) | (df['age'] > 120)]`.
- Format checks: regex for email, phone, or ZIP codes.
- Referential integrity: cross check foreign keys against lookup tables.
 - **Reproducibility:**
- Store cleaning scripts in version control (Git).
- Document assumptions and thresholds in comments or a README.
- Use environment files (`requirements.txt`, `environment.yml`) to lock dependencies.

Examples

Example 1 – Handling Missing Values in a Sales Dataset

```
python
import pandas as pd
from sklearn.impute import SimpleImputer
df = pd.read_csv('sales.csv')
```

1. Identify missingness

```
print(df.isna().sum())
```

2. Impute numeric columns with median

```
num_cols = ['price', 'quantity']
imputer = SimpleImputer(strategy='median')
df[num_cols] = imputer.fit_transform(df[num_cols])
```

3. Impute categorical columns with mode

```
cat_cols = ['product_category']
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```

4. Verify no missing values remain

```
print(df.isna().sum())
```

```
...
```

Explanation:

- The median is robust to outliers, making it suitable for price/quantity.
- Mode fills the most common category, preserving the distribution.
- After imputation, the dataset is ready for aggregation or modeling.

Example 2 – Removing Duplicates and Normalizing Dates

```
python
```

```
import pandas as pd
df = pd.read_csv('orders.csv')
```

Detect duplicates based on order_id and customer_id

```
duplicates = df.duplicated(subset=['order_id', 'customer_id'])
print(f"Found {duplicates.sum()} duplicate rows.")
```

Drop duplicates, keeping the first occurrence

```
df_clean = df.drop_duplicates(subset=['order_id', 'customer_id'], keep='first')
```

Convert order_date to datetime, coerce errors to NaT

```
df_clean['order_date'] = pd.to_datetime(df_clean['order_date'], errors='coerce')
```

Drop rows where date conversion failed

```
df_clean = df_clean.dropna(subset=['order_date'])
print(df_clean.head())
---
```

Explanation:

- `duplicated(subset=...)` focuses on business keys rather than all columns.
- `pd.to_datetime(..., errors='coerce')` turns malformed dates into `NaT`, which can then be removed.
- The resulting DataFrame has unique orders with valid dates, ready for time series analysis.

Quick Review

1. **What is the difference between `df.dropna()` and `df.fillna()`?**
2. **How can you identify duplicate rows that are identical across all columns?**
3. **Why might you choose to convert a column to the `category` dtype in pandas?**
4. **Explain a scenario where dropping rows with missing values is preferable to imputing them.**
5. **What is the purpose of the `errors='coerce'` parameter in `pd.to_datetime()`?**

Further Reading

- **Imputation Techniques** – `scikit-learn`'s `SimpleImputer`, `KNNImputer`, and `IterativeImputer`.
- **Data Validation Libraries** – `pandera`, `great_expectations`.
- **Duplicate Handling in SQL** – `ROW_NUMBER()` window functions for deduplication.
- **Data Cleaning in Big Data** – Spark's `DataFrame.na` methods and `Delta Lake` for versioned datasets.
- **Best Practices** – Writing reproducible cleaning pipelines with `make`, `Snakemake`, or `Airflow`.

